

TALLER EVALUATIVO: AUTOMATIZACIÓN DE INTEGRACIÓN Y ENTREGA CONTINUA (TIEMPO: 2 HORAS)

Ingeniería de Software V

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS
INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

DATOS GENERALES DEL TALLER

REGISTRO

CÓDIGO:	09776 - TIC	PERÍODO:	202620 (Semestre 2026-2)
NRC / GRUPO:	11005 / Grupo 005	PONDERACIÓN:	10.0% (Taller Evaluativo) • Duración: 2 Horas
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	CALIFICACIÓN:	_____ / 5.0
ESTUDIANTE 1:	_____	CÓDIGO:	_____
ESTUDIANTE 2:	_____	CÓDIGO:	_____

1 Contexto Organizacional & Arquitectura del Sistema

1.1 Situación Problema

La empresa **NovaCommerce Andina** atraviesa un proceso de modernización de su plataforma de inventario de productos. Hasta ahora, el equipo de desarrollo entrega el código al equipo de operaciones mediante correos electrónicos y carpetas compartidas, generando fricciones, tiempos muertos y errores de despliegue atribuibles a la falta de un proceso automatizado y de una cultura de trabajo integrada. La plataforma está compuesta por dos componentes desacoplados:

- Backend REST** (`product-api`): Microservicio en **Spring Boot 3 (Java 17)** con JPA y base de datos H2 en memoria, encargado de exponer operaciones CRUD de productos (GET, POST, PUT, DELETE `/api/products`).
- Frontend SPA** (`product-frontend`): Interfaz de usuario en **React 18 + Vite + TypeScript** que consume el API a través de la variable de entorno `VITE_API_URL`.

Como estudiante de Ingeniería de Software, usted es incorporado al equipo de plataforma con un doble objetivo: (i) construir el pipeline técnico de integración y entrega continua sobre el código base suministrado, y (ii) evidenciar, mediante la reflexión de la Sección 3, comprensión de los fundamentos organizacionales de DevOps que sustentan dicha automatización (ruptura de silos, principios culturales y modelo de responsabilidad compartida).

Componente	Tecnología & Stack	Puerto / Endpoint
Backend REST	Java 17, Spring Boot 3.2, Spring Data JPA, H2	Puerto 8080 (<code>/api/products</code>)
Frontend SPA	React 18, TypeScript, Vite 5, Nginx Alpine	Puerto 3000 (Proxy: <code>/</code>)
Orquestación	Docker Compose v2 (Red Bridge <code>product-network</code>)	<code>docker compose up -d</code>
CI/CD Pipeline	GitHub Actions (<code>.github/workflows/ci-cd.yml</code>)	4 Jobs (Test, Lint, Build, Smoke)

2 Desarrollo Guiado: Fases de Implementación

2.1 Fase 1: Verificación de Calidad y Empaquetado de Artefactos (25 min)

Sobre el código base suministrado en `codigo_base/`:

- a) En `backend/`, ejecute la suite de pruebas unitarias y de controlador con Maven:

```
cd backend && mvn clean test
```

Confirme que `ProductServiceTest` y `ProductControllerTest` finalizan sin fallos.

- b) En `frontend/`, valide la consistencia de tipos TypeScript y genere el build de producción:

```
cd frontend && npm install && npm run lint && npm run build
```

2.2 Fase 2: Contenerización Multi-Stage con Docker (35 min)

Diseñe y optimice los archivos `Dockerfile` de cada servicio aplicando buenas prácticas de reducción de peso y seguridad de imagen:

• **Backend** (`backend/Dockerfile`):

- *Etapas 1 (Builder)*: `maven:3.9.6-alpine`. Descarga de dependencias en caché y compilación (`mvn package -DskipTests`).
- *Etapas 2 (Runtime)*: `eclipse-temurin:17-jre-alpine`. Copia del artefacto `.jar`, ejecución con usuario no root (`appuser`) y exposición del puerto 8080.

• **Frontend** (`frontend/Dockerfile`):

- *Etapas 1 (Builder)*: `node:20-alpine`. Instalación limpia (`npm install`) y generación de estáticos (`npm run build`).
- *Etapas 2 (Runtime)*: `nginx:alpine`. Copia de estáticos `dist/` hacia Nginx con soporte de fallback SPA en `nginx.conf`.

2.3 Fase 3: Orquestación Multicontenedor con Docker Compose (20 min)

Complete el archivo `docker-compose.yml` para orquestar la ejecución conjunta de los servicios:

- a) Declare una red compartida tipo puente (`product-network`).
- b) Configure el *healthcheck* del backend validando la respuesta HTTP en `http://localhost:8080/api/products`.
- c) Configure la directiva `depends_on` con `condition: service_healthy` en el contenedor frontend.
- d) Levante el entorno completo y compruebe su funcionamiento en el navegador (`http://localhost:3000`):

```
docker compose up -d --build
docker compose ps
curl -i http://localhost:8080/api/products
```

2.4 Fase 4: Pipeline Declarativo en GitHub Actions (30 min)

Configure el flujo de automatización en `.github/workflows/ci-cd.yml`, estructurado en 4 jobs encadenados:

- a) **Job 1** (`test-backend`): Setup JDK 17 (Temurin), ejecución de `mvn test` y carga del artefacto JAR (`upload-artifact`).
- b) **Job 2** (`test-frontend`): Setup Node 20, verificación estática (`npm run lint`) y compilación del bundle.
- c) **Job 3** (`build-docker`): Construcción de imágenes Docker utilizando `docker/build-push-action` con tags versionados, condicionada al éxito de los Jobs 1 y 2.
- d) **Job 4** (`smoke-test`): Despliegue con `docker compose up -d` y validación de disponibilidad mediante consultas `curl` automatizadas.

3 Reflexión: Fundamentos Culturales y Organizacionales DevOps (10 min)

Responda brevemente cada pregunta relacionando los conceptos teóricos con evidencia concreta del pipeline que usted mismo construyó en las Fases 1 a 4:

1. **Ruptura de Silos Dev-Ops:** En un modelo tradicional, Desarrollo (Dev) y Operaciones (Ops) trabajan en silos organizacionales separados, con objetivos, herramientas e indicadores distintos. Describa dos consecuencias típicas de esta separación en un proyecto de software, y explique cómo la existencia de un único pipeline con jobs interdependientes (`test-backend`, `test-frontend`, `build-docker`, `smoke-test`) contribuye a disolver dicha frontera entre ambos equipos.
2. **Principios CALMS y Decisiones Técnicas:** El modelo CALMS (*Cultura, Automatización, Lean, Medición, Compartir*) resume los pilares culturales de DevOps. Seleccione al menos tres de estos pilares y relaciónelos con decisiones técnicas concretas tomadas durante el taller (por ejemplo, en la contenerización multi-stage o en la automatización del pipeline de GitHub Actions).
3. **Roles y Responsabilidad Compartida (“You Build It, You Run It”):** Explique en qué consiste el principio de responsabilidad compartida entre quien construye el software y quien lo opera en producción, popularizado por Amazon. A partir del Job 4 (`smoke-test`) del pipeline implementado, argumente qué evidencia técnica demuestra que el equipo de desarrollo asume parte de la responsabilidad operativa del sistema, y qué rol cumpliría hoy un ingeniero DevOps frente a este mismo flujo.

4 Rúbrica de Evaluación Modularizada por Punto (Escala 0.0 a 5.0)

Punto Entregable	Pts	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
2.1. Dockerfile Backend	0.5	Multi-stage impecable (Maven builder → JRE 17 slim), usuario no root (appuser) y cache.	Multi-stage funcional pero usa imagen pesada (JDK) o ejecuta como root.	Dockerfile de una sola etapa o genera imágenes de tamaño excesivo (>500MB).
2.2. Dockerfile Frontend	0.5	Multi-stage eficiente (Node → Nginx Alpine), archivo nginx.conf con fallback SPA.	Frontend en Nginx funcional pero sin configuración adecuada de SPA.	Servidor web no levanta o sirve estáticos de forma incorrecta (404 en rutas).
3.1. Docker Compose	1.2	Orquestación completa (product-network), healthchecks condicionales y puertos funcionales.	Stack levanta pero sin healthchecks o dependencias declaradas.	Error al levantar el stack o conflicto de puertos y redes en Docker.
4.1. GitHub Actions CI	1.8	Workflow declarativo con 4 jobs (Test, Lint, Docker Build, Smoke Test) en verde (Passing).	Pipeline funcional pero con jobs secuenciales innecesarios o sin cache.	Pipeline falla, no ejecuta pruebas de humo o sintaxis YAML incorrecta.
5.1. Reflexión DevOps	1.0	Respuestas que articulan con rigor silos, principios CALMS y responsabilidad compartida, conectándolos con evidencia concreta del pipeline construido y bitácora ética de IAG.	Respuestas correctas pero genéricas, con conexión débil entre la teoría y el desarrollo técnico realizado, o bitácora de prompts incompleta.	Respuestas erróneas conceptualmente, sin relación con el pipeline construido o análisis ausente.
CALIFICACIÓN TOTAL	5.0	Escala Institucional Universidad Icesi: Aprobatoria ≥ 3.0 • Duración Máxima: 2 Horas		

INSTRUCCIONES DE ENTREGA

MODALIDAD

- Repositorio en GitHub:** Subir el código base con las soluciones implementadas y verificar que el workflow de **GitHub Actions** esté en Verde (*Passing*).
- Documento de Resultados (PDF):** Adjuntar en la plataforma de entrega este documento diligenciado con:
 - Enlace al repositorio privado en la organización con acceso al docente.
 - Captura de pantalla de la ejecución exitosa de los 4 jobs de GitHub Actions.
 - Respuestas a las tres preguntas de la Sección 3 (Reflexión DevOps).